

我们已经看到了许多无符号运算的细微特性，尤其是有符号数到无符号数的隐式转换，会导致错误或者漏洞的方式。避免这类错误的一种方法就是绝不使用无符号数。实际上，除了 C 以外很少有语言支持无符号整数。很明显，这些语言的设计者认为它们带来的麻烦要比益处多得多。比如，Java 只支持有符号整数，并且要求以补码运算来实现。正常的右移运算符 $\gg$ 被定义为执行算术右移。特殊的运算符 $\ggg$ 被指定为执行逻辑右移。

当我们想要把字仅仅看做是位的集合而没有任何数字意义时，无符号数值是非常有用的。例如，往一个字中放入描述各种布尔条件的标记(flag)时，就是这样。地址自然地就是无符号的，所以系统程序员发现无符号类型是很有帮助的。当实现模运算和多精度运算的数学包时，数字是由字的数组来表示的，无符号值也会非常有用。

2.3 整数运算

许多刚入门的程序员非常惊奇地发现，两个正数相加会得出一个负数，而比较表达式 $x < y$ 和比较表达式 $x - y < 0$ 会产生不同的结果。这些属性是由于计算机运算的有限性造成的。理解计算机运算的细微之处能够帮助程序员编写更可靠的代码。

2.3.1 无符号加法

考虑两个非负整数 x 和 y ，满足 $0 \leq x, y < 2^w$ 。每个数都能表示为 w 位无符号数字。然而，如果计算它们的和，我们就有一个可能的范围 $0 \leq x + y \leq 2^{w+1} - 2$ 。表示这个和可能需要 $w+1$ 位。例如，图 2-21 展示了当 x 和 y 有 4 位表示时，函数 $x+y$ 的坐标图。参数(显示在水平轴上)取值范围为 0~15，但是和的取值范围为 0~30。函数的形状是一个有坡度的平面(在两个维度上，函数都是线性的)。如果保持和为一个 $w+1$ 位的数字，并且把它加上另外一个数值，我们可能需要 $w+2$ 个位，以此类推。这种持续的“字长膨胀”意味着，要想完整地表示算术运算的结果，我们不能对字长做任何限制。一些编程语言，例如 Lisp，实际上就支持无限精度的运算，允许任意的(当然，要在机器的内存限制之内)整数运算。更常见的是，编程语言支持固定精度的运算，因此像“加法”和“乘法”这样的运算不同于它们在整数上的相应运算。

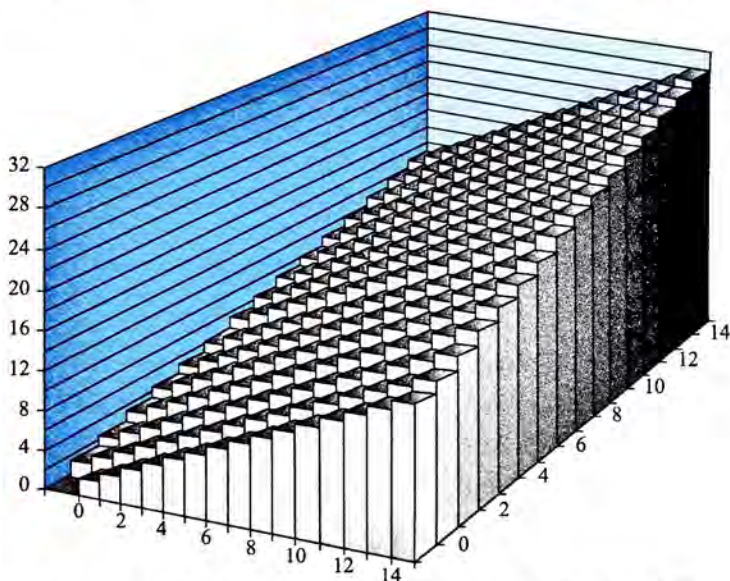


图 2-21 整数加法。对于一个 4 位的字长，其和可能需要 5 位

让我们为参数 x 和 y 定义运算 $+^w$, 其中 $0 \leq x, y < 2^w$, 该操作是把整数和 $x+y$ 截断为 w 位得到的结果, 再把这个结果看做是一个无符号数。这可以被视为一种形式的模运算, 对 $x+y$ 的位级表示, 简单丢弃任何权重大于 2^{w-1} 的位就可以计算出和模 2^w 。比如, 考虑一个 4 位数字表示, $x=9$ 和 $y=12$ 的位表示分别为 $[1001]$ 和 $[1100]$ 。它们的和是 21, 5 位的表示为 $[10101]$ 。但是如果丢弃最高位, 我们就得到 $[0101]$, 也就是说, 十进制值的 5。这就和值 $21 \bmod 16=5$ 一致。

我们可以将操作 $+^w$ 描述为:

原理: 无符号数加法

对满足 $0 \leq x, y < 2^w$ 的 x 和 y 有:

$$x +^w y = \begin{cases} x + y, & x + y < 2^w \\ x + y - 2^w, & 2^w \leq x + y < 2^{w+1} \end{cases} \quad \begin{matrix} \text{正常} \\ \text{溢出} \end{matrix} \quad (2.11)$$

图 2-22 说明了公式(2.11)的这两种情况, 左边的和 $x+y$ 映射到右边的无符号 w 位的和 $x +^w y$ 。正常情况下 $x+y$ 的值保持不变, 而溢出情况则是该和数减去 2^w 的结果。

推导: 无符号数加法

一般而言, 我们可以看到, 如果 $x+y < 2^w$, 和的 $w+1$ 位表示中的最高位会等于 0, 因此丢弃它不会改变这个数值。另一方面, 如果 $2^w \leq x+y < 2^{w+1}$, 和的 $w+1$ 位表示中的最高位会等于 1, 因此丢弃它就相当于从和中减去了 2^w 。



图 2-22 整数加法和无符号加法间的关系。当 $x+y$ 大于 2^w-1 时, 其和溢出

说一个算术运算溢出, 是指完整的整数结果不能放到数据类型的字长限制中去。如等式(2.11)所示, 当两个运算数的和为 2^w 或者更大时, 就发生了溢出。图 2-23 展示了字长 $w=4$ 的无符号加法函数的坐标图。这个和是按模 $2^4=16$ 计算的。当 $x+y < 16$ 时, 没有溢出, 并且 $x +^w y$ 就是 $x+y$ 。这对应于图中标记为“正常”的斜面。当 $x+y \geq 16$ 时, 加法溢出, 结果相当于从和中减去 16。这对应于图中标记为“溢出”的斜面。

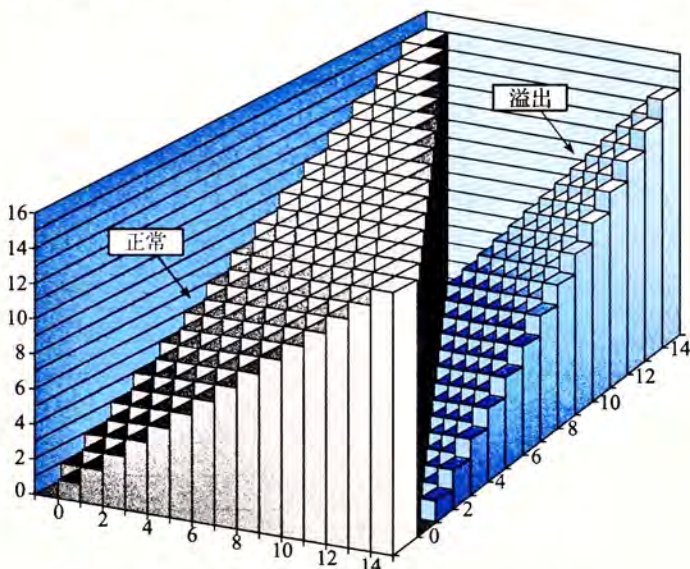


图 2-23 无符号加法(4 位字长, 加法是模 16 的)

当执行 C 程序时, 不会将溢出作为错误而发信号。不过有的时候, 我们可能希望判定是否发生了溢出。

原理: 检测无符号数加法中的溢出

对在范围 $0 \leq x, y \leq UMax_w$ 中的 x 和 y , 令 $s = x + {}^u_w y$ 。则对计算 s , 当且仅当 $s < x$ (或者等价地 $s < y$) 时, 发生了溢出。

作为说明, 在前面的示例中, 我们看到 $9 + {}^u_4 12 = 5$ 。由于 $5 < 9$, 我们可以看出发生了溢出。

推导: 检测无符号数加法中的溢出

通过观察发现 $x + y \geq x$, 因此如果 s 没有溢出, 我们能够肯定 $s \geq x$ 。另一方面, 如果 s 确实溢出了, 我们就有 $s = x + y - 2^w$ 。假设 $y < 2^w$, 我们就有 $y - 2^w < 0$, 因此 $s = x + (y - 2^w) < x$ 。 ■

 **练习题 2.27** 写出一个具有如下原型的函数:

```
/* Determine whether arguments can be added without overflow */
int uadd_ok(unsigned x, unsigned y);
```

如果参数 x 和 y 相加不会产生溢出, 这个函数就返回 1。

模数加法形成了一种数学结构, 称为阿贝尔群 (Abelian group), 这是以丹麦数学家 Niels Henrik Abel (1802~1829) 的名字命名。也就是说, 它是可交换的 (这就是为什么叫 “abelian” 的地方) 和可结合的。它有一个单位元 0, 并且每个元素有一个加法逆元。让我们考虑 w 位的无符号数的集合, 执行加法运算 $+^u_w$ 。对于每个值 x , 必然有某个值 $-^u_w x$ 满足 $-^u_w x + {}^u_w x = 0$ 。该加法的逆操作可以表述如下:

原理: 无符号数求反

对满足 $0 \leq x < 2^w$ 的任意 x , 其 w 位的无符号逆元 $-^u_w x$ 由下式给出:

$$-^u_w x = \begin{cases} x, & x = 0 \\ 2^w - x, & x > 0 \end{cases} \quad (2.12)$$

该结果可以很容易地通过案例分析推导出来:

推导: 无符号数求反

当 $x = 0$ 时, 加法逆元显然是 0。对于 $x > 0$, 考虑值 $2^w - x$ 。我们观察到这个数字在 $0 < 2^w - x < 2^w$ 范围之内, 并且 $(x + 2^w - x) \bmod 2^w = 2^w \bmod 2^w = 0$ 。因此, 它就是 x 在 $+^u_w$ 下的逆元。 ■

 **练习题 2.28** 我们能用一个十六进制数字来表示长度 $w=4$ 的位模式。对于这些数字的无符号解释, 使用等式 (2.12) 填写下表, 给出所示数字的无符号加法逆元的位表示 (用十六进制形式)。

x		$-^u_4 x$	
十六进制	十进制	十进制	十六进制
0			
5			
8			
D			
F			

2.3.2 补码加法

对于补码加法, 我们必须确定当结果太大 (为正) 或者太小 (为负) 时, 应该做些什么。

给定在范围 $-2^{w-1} \leq x, y \leq 2^{w-1}-1$ 之内的整数值 x 和 y , 它们的和就在范围 $-2^w \leq x+y \leq 2^w-2$ 之内, 要想准确表示, 可能需要 $w+1$ 位。就像以前一样, 我们通过将表示截断到 w 位, 来避免数据大小的不断扩张。然而, 结果却不像模数加法那样在数学上感觉很熟悉。定义 $x+_w y$ 为整数和 $x+y$ 被截断为 w 位的结果, 并将这个结果看做是补码数。

原理: 补码加法

对满足 $-2^{w-1} \leq x, y \leq 2^{w-1}-1$ 的整数 x 和 y , 有:

$$x+_w y = \begin{cases} x+y-2^w, & 2^{w-1} \leq x+y \\ x+y, & -2^{w-1} \leq x+y < 2^{w-1} \\ x+y+2^w, & x+y < -2^{w-1} \end{cases} \quad \begin{matrix} \text{正溢出} \\ \text{正常} \\ \text{负溢出} \end{matrix} \quad (2.13)$$

图 2-24 说明了这个原理, 其中, 左边的和 $x+y$ 的取值范围为 $-2^w \leq x+y \leq 2^w-2$, 右边显示的是该和数截断为 w 位补码的结果。(图中的标号“情况 1”到“情况 4”用于该原理形式化推导的案例中。) 当和 $x+y$ 超过 $TMax_w$ 时(情况 4), 我们说发生了正溢出。在这种情况下, 截断的结果是从和数中减去 2^w 。当和 $x+y$ 小于 $TMin_w$ 时(情况 1), 我们说发生了负溢出。在这种情况下, 截断的结果是把和数加上 2^w 。

两个数的 w 位补码之和与无符号之和有完全相同的位级表示。实际上, 大多数计算机使用同样的机器指令来执行无符号或者有符号加法。

推导: 补码加法

既然补码加法与无符号数加法有相同的位级表示, 我们就可以按如下步骤表示运算 $+_w$: 将其参数转换为无符号数, 执行无符号数加法, 再将结果转换为补码:

$$x+_w y \doteq U2T_w(T2U_w(x) +_w T2U_w(y)) \quad (2.14)$$

根据等式(2.6), 我们可以把 $T2U_w(x)$ 写成 $x_{w-1}2^w + x$, 把 $T2U_w(y)$ 写成 $y_{w-1}2^w + y$ 。使用属性, 即 $+_w$ 是模 2^w 的加法, 以及模数加法的属性, 我们就能得到:

$$\begin{aligned} x+_w y &= U2T_w(T2U_w(x) +_w T2U_w(y)) \\ &= U2T_w[(x_{w-1}2^w + x + y_{w-1}2^w + y) \bmod 2^w] \\ &= U2T_w[(x+y) \bmod 2^w] \end{aligned}$$

消除了 $x_{w-1}2^w$ 和 $y_{w-1}2^w$ 这两项, 因为它们模 2^w 等于 0。

为了更好地理解这个数量, 定义 z 为整数和 $z \doteq x+y$, z' 为 $z' \doteq z \bmod 2^w$, 而 z'' 为 $z'' \doteq U2T_w(z')$ 。数值 z'' 等于 $x+_w y$ 。我们分成 4 种情况分析, 如图 2-24 所示。

1) $-2^w \leq z < -2^{w-1}$ 。然后, 我们会有 $z' = z + 2^w$ 。这就得出 $0 \leq z' < -2^{w-1} + 2^w = 2^{w-1}$ 。检查等式(2.7), 我们看到 z' 在满足 $z'' = z'$ 的范围之内。这种情况称为负溢出(negative overflow)。我们将两个负数 x 和 y 相加(这是我们能得到 $z < -2^{w-1}$ 的唯一方式), 得到一个非负的结果 $z'' = x+y+2^w$ 。

2) $-2^{w-1} \leq z < 0$ 。那么, 我们又将有 $z' = z$, 得到 $-2^{w-1} \leq z' < 2^w$ 。检查等式(2.7), 我们看到 z' 在满足 $z'' = z' - 2^w$ 的范围之内, 因此 $z'' = z' - 2^w = z + 2^w - 2^w = z$ 。也就是说, 我们的补码和 z'' 等于整数和 $x+y$ 。

3) $0 \leq z < 2^{w-1}$ 。那么, 我们将有 $z' = z$, 得到 $0 \leq z' < 2^{w-1}$, 因此 $z'' = z' = z$ 。补码和

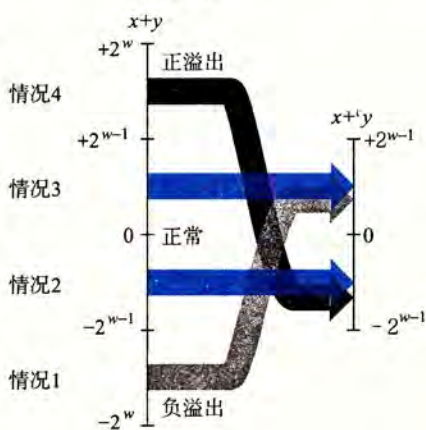


图 2-24 整数和补码加法之间的关系。当 $x+y$ 小于 -2^{w-1} 时, 产生负溢出。当它大于 2^{w-1} 时, 产生正溢出

z'' 又等于整数和 $x+y$ 。

4) $2^{w-1} \leq z < 2^w$ 。我们又将有 $z' = z$ ，得到 $2^{w-1} \leq z' < 2^w$ 。但是在这个范围内，我们有 $z'' = z' - 2^w$ ，得到 $z'' = x + y - 2^w$ 。这种情况称为正溢出(positive overflow)。我们将正数 x 和 y 相加(这是我们能得到 $z \geq 2^{w-1}$ 的唯一方式)，得到一个负数结果 $z'' = x + y - 2^w$ 。 ■

图 2-25 展示了一些 4 位补码加法的示例作为说明。每个示例的情况都被标号为对应于等式(2.13)的推导过程中的情况。注意 $2^4 = 16$ ，因此负溢出得到的结果比整数和大 16，而正溢出得到的结果比之小 16。我们包括了运算数和结果的位级表示。可以观察到，能够通过对运算数执行二进制加法并将结果截断到 4 位，从而得到结果。

x	y	$x + y$	$x + \frac{1}{4}y$	情况
-8	-5	-13	3	1
[1000]	[1011]	[10011]	[0011]	
-8	-8	-16	0	1
[1000]	[1000]	[10000]	[0000]	
-8	5	-3	-3	2
[1000]	[0101]	[11101]	[1101]	
2	5	7	7	3
[0010]	[0101]	[00111]	[0111]	
5	5	10	-6	4
[0101]	[0101]	[01010]	[1010]	

图 2-25 补码加法示例。通过执行运算数的二进制加法并将结果截断到 4 位，可以获得 4 位补码和的位级表示

图 2-26 阐述了字长 $w=4$ 的补码加法。运算数的范围为 $-8 \sim 7$ 之间。当 $x+y < -8$ 时，补码加法就会负溢出，导致和增加了 16。当 $-8 \leq x+y < 8$ 时，加法就产生 $x+y$ 。当 $x+y \geq 8$ ，加法就会正溢出，使得和减少了 16。这三种情况中的每一种都形成了图中的一个斜面。

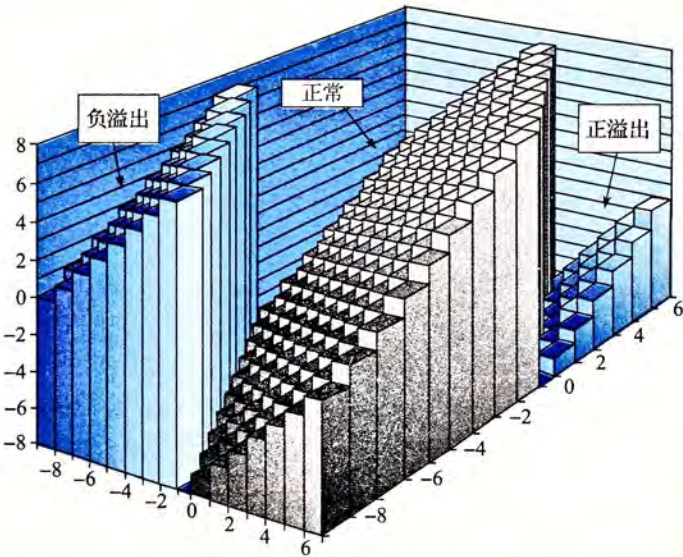


图 2-26 补码加法(字长为 4 位的情况下，当 $x+y < -8$ 时，产生负溢出； $x+y \geq 8$ 时，产生正溢出)

等式(2.13)也让我们认出了哪些情况下会发生溢出:

原理: 检测补码加法中的溢出

对满足 $TMin_w \leq x, y \leq TMax_w$ 的 x 和 y , 令 $s \doteq x + {}^w_2y$ 。当且仅当 $x > 0, y > 0$, 但 $s \leq 0$ 时, 计算 s 发生了正溢出。当且仅当 $x < 0, y < 0$, 但 $s \geq 0$ 时, 计算 s 发生了负溢出。

图 2-25 显示了当 $w=4$ 时, 这个原理的例子。第一个条目是负溢出的情况, 两个负数相加得到一个正数。最后一个条目是正溢出的情况, 两个正数相加得到一个负数。

推导: 检测补码加法中的溢出

让我们先来分析正溢出。如果 $x > 0, y > 0$, 而 $s \leq 0$, 那么显然发生了正溢出。反过来, 正溢出的条件为: 1) $x > 0, y > 0$ (或者 $x + y < TMax_w$), 2) $s \leq 0$ (见公式(2.13))。同样的讨论也适用于负溢出情况。 ■

 **练习题 2.29** 按照图 2-25 的形式填写下表。分别列出 5 位参数的整数值、整数和与补码和的数值、补码和的位级表示, 以及属于等式(2.13)推导中的哪种情况。

x	y	$x+y$	$x+{}^5_2y$	情况
[10100]	[10001]			
[11000]	[11000]			
[10111]	[01000]			
[00010]	[00101]			
[01100]	[00100]			

 **练习题 2.30** 写出一个具有如下原型的函数:

```
/* Determine whether arguments can be added without overflow */
int tadd_ok(int x, int y);
```

如果参数 x 和 y 相加不会产生溢出, 这个函数就返回 1。

 **练习题 2.31** 你的同事对你补码加法溢出条件的分析有些不耐烦了, 他给出了一个函数 `tadd_ok` 的实现, 如下所示:

```
/* Determine whether arguments can be added without overflow */
/* WARNING: This code is buggy. */
int tadd_ok(int x, int y) {
    int sum = x+y;
    return (sum-x == y) && (sum-y == x);
}
```

你看了代码以后笑了。解释一下为什么。

 **练习题 2.32** 你现在有个任务, 编写函数 `tsub_ok` 的代码, 函数的参数是 x 和 y , 如果计算 $x-y$ 不产生溢出, 函数就返回 1。假设你写的练习题 2.30 的代码如下所示:

```
/* Determine whether arguments can be subtracted without overflow */
/* WARNING: This code is buggy. */
int tsub_ok(int x, int y) {

    return tadd_ok(x, -y);
}
```

x 和 y 取什么值时, 这个函数会产生错误的结果? 写一个该函数的正确版本(家庭作业 2.74)。

2.3.3 补码的非

可以看到范围在 $TMin_w \leq x \leq TMax_w$ 中的每个数字 x 都有 $+_w^1$ 下的加法逆元, 我们将 $-_w^1 x$ 表示如下。

原理: 补码的非

对满足 $TMin_w \leq x \leq TMax_w$ 的 x , 其补码的非 $-_w^1 x$ 由下式给出

$$-_w^1 x = \begin{cases} TMin_w, & x = TMin_w \\ -x, & x > TMin_w \end{cases} \quad (2.15)$$

也就是说, 对 w 位的补码加法来说, $TMin_w$ 是自己的加法的逆, 而对其他任何数值 x 都有 $-x$ 作为其加法的逆。

推导: 补码的非

观察发现 $TMin_w + TMin_w = -2^{w-1} + (-2^{w-1}) = -2^w$ 。这将导致负溢出, 因此 $TMin_w + {}_w^1 TMin_w = -2^w + 2^w = 0$ 。对满足 $x > TMin_w$ 的 x , 数值 $-x$ 可以表示为一个 w 位的补码, 它们的和 $-x + x = 0$ 。 ■

 **练习题 2.33** 我们可以用一个十六进制数字来表示长度 $w=4$ 的位模式。根据这些数字的补码的解释, 填写下表, 确定所示数字的加法逆元。

x		$-_w^1 x$	
十六进制	十进制	十进制	十六进制
0			
5			
8			
D			
F			

对于补码和无符号(练习题 2.28)非(negation)产生的位模式, 你观察到什么?

网络旁注 DATA: TNEG 补码非的位级表示

计算一个位级表示的值的补码非有几种聪明的方法。这些技术很有用(例如当你在调试程序的时候遇到值 `0xfffffffffa`), 同时它们也能够让你更了解补码表示的本质。

执行位级补码非的第一种方法是对每一位求补, 再对结果加 1。在 C 语言中, 我们可以说, 对于任意整数 x , 计算表达式 $-x$ 和 $\sim x + 1$ 得到的结果完全一样。

下面是一些示例, 字长为 4:

$\vec{x}$		$\sim \vec{x}$		$incr(\sim \vec{x})$	
[0101]	5	[1010]	-6	[1011]	-5
[0111]	7	[1000]	-8	[1001]	-7
[1100]	-4	[0011]	3	[0100]	4
[0000]	0	[1111]	-1	[0000]	0
[1000]	-8	[0111]	7	[1000]	-8

从前面的例子我们知道 `0xf` 的补是 `0x0`, 而 `0xa` 的补是 `0x5`, 因而 `0xfffffffffa` 是 -6 的补码表示。

计算一个数 x 的补码非的第二种方法是建立在将位向量分为两部分的基础之上的。假设 k 是最右边的 1 的位置, 因而 x 的位级表示形如 $[x_{w-1}, x_{w-2}, \dots, x_{k+1}, 1, 0, \dots, 0]$ 。(只要 $x \neq 0$ 就能够找到这样的 k 。)这个值的非写成二进制格式就是 $[\sim x_{w-1}, \sim x_{w-2}, \dots,$

$\sim x_{k+1}, 1, 0, \dots, 0]$ 。也就是，我们对位 k 左边的所有位取反。

我们用一些 4 位数字来说明这个方法，这里我们用斜体来突出最右边的模式 1, 0, ..., 0:

x		$-x$	
[1100]	-4	[0100]	4
[1000]	-8	[1000]	-8
[0101]	5	[1011]	-5
[0111]	7	[1001]	-7

2.3.4 无符号乘法

范围在 $0 \leq x, y \leq 2^w - 1$ 内的整数 x 和 y 可以被表示为 w 位的无符号数，但是它们的乘积 $x \cdot y$ 的取值范围为 0 到 $(2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$ 之间。这可能需要 $2w$ 位来表示。不过，C 语言中的无符号乘法被定义为产生 w 位的值，就是 $2w$ 位的整数乘积的低 w 位表示的值。我们将这个值表示为 $x *^u_w y$ 。

将一个无符号数截断为 w 位等价于计算该值模 2^w ，得到：

原理：无符号数乘法

对满足 $0 \leq x, y \leq UMax_w$ 的 x 和 y 有：

$$x *^u_w y = (x \cdot y) \bmod 2^w \quad (2.16)$$

2.3.5 补码乘法

范围在 $-2^{w-1} \leq x, y \leq 2^{w-1} - 1$ 内的整数 x 和 y 可以被表示为 w 位的补码数字，但是它们的乘积 $x \cdot y$ 的取值范围为 $-2^{w-1} \cdot (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$ 到 $-2^{w-1} \cdot -2^{w-1} = -2^{2w-2}$ 之间。要想用补码来表示这个乘积，可能需要 $2w$ 位。然而，C 语言中的有符号乘法是通过将 $2w$ 位的乘积截断为 w 位来实现的。我们将这个数值表示为 $x *^s_w y$ 。将一个补码数截断为 w 位相当于先计算该值模 2^w ，再把无符号数转换为补码，得到：

原理：补码乘法

对满足 $TMin_w \leq x, y \leq TMax_w$ 的 x 和 y 有：

$$x *^s_w y = U2T_w((x \cdot y) \bmod 2^w) \quad (2.17)$$

我们认为对于无符号和补码乘法来说，乘法运算的位级表示都是一样的，并用如下原理说明：

原理：无符号和补码乘法的位级等价性

给定长度为 w 的位向量 $\vec{x}$ 和 $\vec{y}$ ，用补码形式的位向量表示来定义整数 x 和 y ： $x = B2T_w(\vec{x})$, $y = B2T_w(\vec{y})$ 。用无符号形式的位向量表示来定义非负整数 x' 和 y' ： $x' = B2U_w(\vec{x})$, $y' = B2U_w(\vec{y})$ 。则

$$T2B_w(x *^s_w y) = U2B_w(x' *^u_w y')$$

作为说明，图 2-27 给出了不同 3 位数字的乘法结果。对于每一对位级运算数，我们执行无符号和补码乘法，得到 6 位的乘积，然后再把这些乘积截断到 3 位。无符号的截断后的乘积总是等于 $x \cdot y \bmod 8$ 。虽然无符号和补码两种乘法乘积的 6 位表示不同，但是截断后的乘积的位级表示都相同。

推导：无符号和补码乘法的位级等价性

根据等式(2.6)，我们有 $x' = x + x_{w-1}2^w$ 和 $y' = y + y_{w-1}2^w$ 。计算这些值的乘积模 2^w 得到以下结果：

$$(x' \cdot y') \bmod 2^w = [(x + x_{w-1}2^w) \cdot (y + y_{w-1}2^w)] \bmod 2^w$$

$$\begin{aligned} &= [x \cdot y + (x_{w-1}y + y_{w-1}x)2^w + x_{w-1}y_{w-1}2^{2w}] \bmod 2^w \\ &= (x \cdot y) \bmod 2^w \end{aligned} \tag{2.18}$$

由于模运算符，所有带有权重 2^w 和 2^{2w} 的项都丢掉了。根据等式(2.17)，我们有 $x *'_w y = U2T_w((x \cdot y) \bmod 2^w)$ 。对等式两边应用操作 $T2U_w$ 有：

$$T2U_w(x *'_w y) = T2U_w(U2T_w((x \cdot y) \bmod 2^w)) = (x \cdot y) \bmod 2^w$$

将上述结果与式(2.16)和式(2.18)结合起来得到 $T2U_w(x *'_w y) = (x' \cdot y') \bmod 2^w = x' *'_w y'$ 。然后对这个等式的两边应用 $U2B_w$ ，得到

$$U2B_w(T2U_w(x *'_w y)) = T2B_w(x *'_w y) = U2B_w(x' *'_w y') \quad \blacksquare$$

模式	x		y		x·y		截断的x·y	
无符号	5	[101]	3	[011]	15	[001111]	7	[111]
补码	-3	[101]	3	[011]	-9	[110111]	-1	[111]
无符号	4	[100]	7	[111]	28	[011100]	4	[100]
补码	-4	[100]	-1	[111]	4	[000100]	-4	[100]
无符号	3	[011]	3	[011]	9	[001001]	1	[001]
补码	3	[011]	3	[011]	9	[001001]	1	[001]

图 2-27 3 位无符号和补码乘法示例。虽然完整的乘积的位级表示可能会不同，但是截断后乘积的位级表示是相同的

 **练习题 2.34** 按照图 2-27 的风格填写下表，说明不同的 3 位数字乘法的结果。

模式	x	y	x·y	截断的x·y
无符号	[100]	[101]		
补码	[100]	[101]		
无符号	[010]	[111]		
补码	[010]	[111]		
无符号	[110]	[110]		
补码	[110]	[110]		

 **练习题 2.35** 给你一个任务，开发函数 `tmult_ok` 的代码，该函数会判断两个参数相乘是否会产生溢出。下面是你的解决方案：

```
/* Determine whether arguments can be multiplied without overflow */
int tmult_ok(int x, int y) {
    int p = x*y;
    /* Either x is zero, or dividing p by x gives y */
    return !x || p/x == y;
}
```

你用 x 和 y 的很多值来测试这段代码，似乎都工作正常。你的同事挑战你，说：“如果我不能用减法来检验加法是否溢出(参见练习题 2.31)，那么你怎么能用除法来检验乘法是否溢出呢？”

按照下面的思路，用数学推导来证明你的方法是对的。首先，证明 $x=0$ 的情况是正确的。另外，考虑 w 位数字 $x(x \neq 0)$ 、 y 、 p 和 q ，这里 p 是 x 和 y 补码乘法的结果，而 q 是 p 除以 x 的结果。

1) 说明 x 和 y 的整数乘积 $x \cdot y$ ，可以写成这样的形式： $x \cdot y = p + t2^w$ ，其中， $t \neq 0$ 当且仅当 p 的计算溢出。

2) 说明 p 可以写成这样的形式: $p = x \cdot q + r$, 其中 $|r| < |x|$ 。

3) 说明 $q = y$ 当且仅当 $r = t = 0$ 。

 **练习题 2.36** 对于数据类型 `int` 为 32 位的情况, 设计一个版本的 `tmult_ok` 函数(练习题 2.35), 使用 64 位精度的数据类型 `int64_t`, 而不使用除法。

旁注 XDR 库中的安全漏洞

2002 年, 人们发现 Sun Microsystems 公司提供的实现 XDR 库的代码有安全漏洞, XDR 库是一个广泛使用的、程序间共享数据结构的工具, 造成这个安全漏洞的原因是程序会在毫无察觉的情况下产生乘法溢出。

包含安全漏洞的代码与下面所示类似:

```

1  /* Illustration of code vulnerability similar to that found in
2   * Sun's XDR library.
3   */
4  void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size) {
5      /*
6       * Allocate buffer for ele_cnt objects, each of ele_size bytes
7       * and copy from locations designated by ele_src
8       */
9      void *result = malloc(ele_cnt * ele_size);
10     if (result == NULL)
11         /* malloc failed */
12         return NULL;
13     void *next = result;
14     int i;
15     for (i = 0; i < ele_cnt; i++) {
16         /* Copy object i to destination */
17         memcpy(next, ele_src[i], ele_size);
18         /* Move pointer to next memory region */
19         next += ele_size;
20     }
21     return result;
22 }
```

函数 `copy_elements` 设计用来将 `ele_cnt` 个数据结构复制到第 9 行的函数分配的缓冲区中, 每个数据结构包含 `ele_size` 个字节。需要的字节数是通过计算 `ele_cnt * ele_size` 得到的。

想象一下, 一个怀有恶意的程序员在被编译为 32 位的程序中用参数 `ele_cnt` 等于 $1\,048\,577(2^{20}+1)$ 、`ele_size` 等于 $4096(2^{12})$ 来调用这个函数。然后第 9 行上的乘法会溢出, 导致只会分配 4096 个字节, 而不是装下这些数据所需要的 $4\,294\,971\,392$ 个字节。从第 15 行开始的循环会试图复制所有的字节, 超越已分配的缓冲区的界限, 因而破坏了其他的数据结构。这会导致程序崩溃或者行为异常。

几乎每个操作系统都使用了这段 Sun 的代码, 像 Internet Explorer 和 Kerberos 验证系统这样使用广泛的程序都用到了它。计算机紧急响应组 (Computer Emergency Response Team, CERT), 由卡内基-梅隆软件工程协会 (Carnegie Mellon Software Engineering Institute) 运作的一个追踪安全漏洞或失效的组织, 发布了建议 “CA-2002-25”, 于是许多公司急忙对它们的代码打补丁。幸运的是, 还没有由于这个漏洞引起的安全失效的报告。

库函数 `calloc` 的实现中存在着类似的漏洞。这些已经被修补过了。遗憾的是, 许

多程序员调用分配函数(如 malloc)时,使用算术表达式作为参数,并且不对这些表达式进行溢出检查。编写 calloc 的可靠版本留作一道练习题(家庭作业 2.76)。

 **练习题 2.37** 现在你有一个任务,当数据类型 int 和 size_t 都是 32 位的,修补上述旁注给出的 XDR 代码中的漏洞。你决定将待分配字节数设置为数据类型 uint64_t,来消除乘法溢出的可能性。你把原来对 malloc 函数的调用(第 9 行)替换如下:

```
uint64_t asize =
    ele_cnt * (uint64_t) ele_size;
void *result = malloc(asize);
```

提醒一下, malloc 的参数类型是 size_t。

A. 这段代码对原始的代码有了哪些改进?

B. 你该如何修改代码来消除这个漏洞?

2.3.6 乘以常数

以往,在大多数机器上,整数乘法指令相当慢,需要 10 个或者更多的时钟周期,然而其他整数运算(例如加法、减法、位级运算和移位)只需要 1 个时钟周期。即使在我们的参考机器 Intel Core i7 Haswell 上,其整数乘法也需要 3 个时钟周期。因此,编译器使用了一项重要的优化,试着用移位和加法运算的组合来代替乘以常数因子的乘法。首先,我们会考虑乘以 2 的幂的情况,然后再概括成乘以任意常数。

原理: 乘以 2 的幂

设 x 为位模式 $[x_{w-1}, x_{w-2}, \dots, x_0]$ 表示的无符号整数。那么,对于任何 $k \geq 0$,我们都认为 $[x_{w-1}, x_{w-2}, \dots, x_0, 0, \dots, 0]$ 给出了 $x2^k$ 的 $w+k$ 位的无符号表示,这里右边增加了 k 个 0。

因此,比如,当 $w=4$ 时,11 可以被表示为 $[1011]$ 。 $k=2$ 时将其左移得到 6 位向量 $[101100]$,即可编码为无符号数 $11 \cdot 4=44$ 。

推导: 乘以 2 的幂

这个属性可以通过等式(2.1)推导出来:

$$\begin{aligned} B2U_{w+k}([x_{w-1}, x_{w-2}, \dots, x_0, 0, \dots, 0]) &= \sum_{i=0}^{w-1} x_i 2^{i+k} \\ &= \left[\sum_{i=0}^{w-1} x_i 2^i \right] \cdot 2^k \\ &= x2^k \end{aligned}$$

当对固定字长左移 k 位时,其高 k 位被丢弃,得到

$$[x_{w-k-1}, x_{w-k-2}, \dots, x_0, 0, \dots, 0]$$

而执行固定字长的乘法也是这种情况。因此,我们可以看出左移一个数值等价于执行一个与 2 的幂相乘的无符号乘法。

原理: 与 2 的幂相乘的无符号乘法

C 变量 x 和 k 有无符号数值 x 和 k , 且 $0 \leq k < w$, 则 C 表达式 $x \ll k$ 产生数值 $x * {}_w^u 2^k$ 。

由于固定大小的补码算术运算的位级操作与其无符号运算等价,我们就可以对补码运算的 2 的幂的乘法与左移之间的关系进行类似的表述:

原理: 与 2 的幂相乘的补码乘法

C 变量 x 和 k 有补码值 x 和无符号数值 k , 且 $0 \leq k < w$, 则 C 表达式 $x \ll k$ 产生数值 $x * {}_w^i 2^k$ 。

注意，无论是无符号运算还是补码运算，乘以 2 的幂都可能会导致溢出。结果表明，即使溢出的时候，我们通过移位得到的结果也是一样的。回到前面的例子，我们将 4 位模式 [1011] (数值为 11) 左移两位得到 [101100] (数值为 44)。将这个值截断为 4 位得到 [1100] (数值为 $12 = 44 \bmod 16$)。

由于整数乘法比移位和加法的代价要大得多，许多 C 语言编译器试图以移位、加法和减法的组合来消除很多整数乘以常数的情况。例如，假设一个程序包含表达式 $x * 14$ 。利用 $14 = 2^3 + 2^2 + 2^1$ ，编译器会将乘法重写为 $(x \ll 3) + (x \ll 2) + (x \ll 1)$ ，将一个乘法替换为三个移位和两个加法。无论 x 是无符号的还是补码，甚至当乘法会导致溢出时，两个计算都会得到一样的结果。(根据整数运算的属性可以证明这一点。)更好的是，编译器还可以利用属性 $14 = 2^4 - 2^1$ ，将乘法重写为 $(x \ll 4) - (x \ll 1)$ ，这时只需要两个移位和一个减法。

 **练习题 2.38** 就像我们将在第 3 章中看到的那样，LEA 指令能够执行形如 $(a \ll k) + b$ 的计算，这里 k 等于 0、1、2 或 3，而 b 等于 0 或者某个程序值。编译器常常用这条指令来执行常数因子乘法。例如，我们可以用 $(a \ll 1) + a$ 来计算 $3 * a$ 。

考虑 b 等于 0 或者等于 a 、 k 为任意可能的值的情况，用一条 LEA 指令可以计算 a 的哪些倍数？

归纳一下我们的例子，考虑一个任务，对于某个常数 K 的表达式 $x * K$ 生成代码。编译器会将 K 的二进制表示表达为一组 0 和 1 交替的序列：

$$[(0 \cdots 0)(1 \cdots 1)(0 \cdots 0) \cdots (1 \cdots 1)]$$

例如，14 可以写成 $[(0 \cdots 0)(111)(0)]$ 。考虑一组从位位置 n 到位位置 m 的连续的 $1 (n \geq m)$ 。(对于 14 来说，我们有 $n=3$ 和 $m=1$ 。)我们可以用下面两种不同形式中的一种来计算这些位对乘积的影响：

形式 A: $(x \ll n) + (x \ll (n-1)) + \cdots + (x \ll m)$

形式 B: $(x \ll (n+1)) - (x \ll m)$

把每个这样连续的 1 的结果加起来，不用做任何乘法，我们就能计算出 $x * K$ 。当然，选择使用移位、加法和减法的组合，还是使用一条乘法指令，取决于这些指令的相对速度，而这些是与机器高度相关的。大多数编译器只在需要少量移位、加法和减法就足够的时候才使用这种优化。

 **练习题 2.39** 对于位位置 n 为最高有效位的情况，我们要怎样修改形式 B 的表达式？

 **练习题 2.40** 对于下面每个 K 的值，找出只用指定数量的运算表达 $x * K$ 的方法，这里我们认为加法和减法的开销相当。除了我们已经考虑过的简单的形式 A 和 B 原则，你可能会需要使用一些技巧。

K	移位	加法/减法	表达式
6	2	1	
31	1	1	
-6	2	1	
55	2	2	

 **练习题 2.41** 对于一组从位位置 n 开始到位位置 m 的连续的 $1 (n \geq m)$ ，我们看到可以产生两种形式的代码，A 和 B。编译器该如何决定使用哪一种呢？

2.3.7 除以 2 的幂

在大多数机器上，整数除法要比整数乘法更慢——需要 30 个或者更多的时钟周期。

除以 2 的幂也可以用移位运算来实现,只不过我们用的是右移,而不是左移。无符号和补码数分别使用逻辑移位和算术移位来达到目的。

整数除法总是舍入到零。为了准确进行定义,我们要引入一些符号。对于任何实数 a , 定义 $\lfloor a \rfloor$ 为唯一的整数 a' , 使得 $a' \leq a < a' + 1$ 。例如, $\lfloor 3.14 \rfloor = 3$, $\lfloor -3.14 \rfloor = -4$ 而 $\lceil 3 \rceil = 3$ 。同样, 定义 $\lceil a \rceil$ 为唯一的整数 a' , 使得 $a' - 1 < a \leq a'$ 。例如, $\lceil 3.14 \rceil = 4$, $\lceil -3.14 \rceil = -3$, 而 $\lceil 3 \rceil = 3$ 。对于 $x \geq 0$ 和 $y > 0$, 结果会是 $\lfloor x/y \rfloor$, 而对于 $x < 0$ 和 $y > 0$, 结果会是 $\lceil x/y \rceil$ 。也就是说, 它将向下舍入一个正值, 而向上舍入一个负值。

对无符号运算使用移位是非常简单的, 部分原因是由于无符号数的右移一定是逻辑右移。

原理: 除以 2 的幂的无符号除法

C 变量 x 和 k 有无符号数值 x 和 k , 且 $0 \leq k < w$, 则 C 表达式 $x \gg k$ 产生数值 $\lfloor x/2^k \rfloor$ 。

例如, 图 2-28 给出了在 12 340 的 16 位表示上执行逻辑右移的结果, 以及对它执行除以 1、2、16 和 256 的结果。从左端移入的 0 以斜体表示。我们还给出了用真正的运算做除法得到的结果。这些示例说明, 移位总是舍入到零的结果, 这一点与整数除法的规则一样。

k	$\gg k$ (二进制)	十进制	$12340/2^k$
0	0011000000110100	12340	12340.0
1	0001100000011010	6170	6170.0
4	0000001100000011	771	771.25
8	000000000110000	48	48.203125

图 2-28 无符号数除以 2 的幂(这个例子说明了执行一个逻辑右移 k 位与除以 2^k 再舍入到零有一样的效果)

推导: 除以 2 的幂的无符号除法

设 x 为位模式 $[x_{w-1}, x_{w-2}, \dots, x_0]$ 表示的无符号整数, 而 k 的取值范围为 $0 \leq k < w$ 。设 x' 为 $w-k$ 位位表示 $[x_{w-1}, x_{w-2}, \dots, x_k]$ 的无符号数, 而 x'' 为 k 位位表示 $[x_{k-1}, \dots, x_0]$ 的无符号数。由此, 我们可以看到 $x = 2^k x' + x''$, 而 $0 \leq x'' < 2^k$ 。因此, 可得 $\lfloor x/2^k \rfloor = x'$ 。

对位向量 $[x_{w-1}, x_{w-2}, \dots, x_0]$ 逻辑右移 k 位会得到位向量

$$[0, \dots, 0, x_{w-1}, x_{w-2}, \dots, x_k]$$

这个位向量有数值 x' , 我们看到, 该值可以通过计算 $x \gg k$ 得到。

对于除以 2 的幂的补码运算来说, 情况要稍微复杂一些。首先, 为了保证负数仍然为负, 移位要执行的是算术右移。现在让我们来看看这种右移会产生什么结果。

原理: 除以 2 的幂的补码除法, 向下舍入

C 变量 x 和 k 分别有补码值 x 和无符号数值 k , 且 $0 \leq k < w$, 则当执行算术移位时, C 表达式 $x \ggg k$ 产生数值 $\lfloor x/2^k \rfloor$ 。

对于 $x \geq 0$, 变量 x 的最高有效位为 0, 所以效果与逻辑右移是一样的。因此, 对于非负数来说, 算术右移 k 位与除以 2^k 是一样的。作为一个负数的例子, 图 2-29 给出了对 -12 340 的 16 位表示进行算术右移不同位数的结果。对于不需要舍入的情况 ($k=1$), 结果是 $x/2^k$ 。但是当需要进行舍入时, 移位导致结果向下舍入。例如, 右移 4 位将会把 -771.25 向下舍入为 -772。我们需要调整策略来处理负数 x 的除法。

k	>> k (二进制)	十进制	$-12340/2^k$
0	1100111111001100	-12340	-12340.0
1	1110011111100110	-6170	-6170.0
4	1111100111111100	-772	-771.25
8	1111111110011111	-49	-48.203125

图 2-29 进行算术右移(这个例子说明了算术右移类似于除以 2 的幂，除了是向下舍入，而不是向零舍入)

推导：除以 2 的幂的补码除法，向下舍入

设 x 为位模式 $[x_{w-1}, x_{w-2}, \dots, x_0]$ 表示的补码整数，而 k 的取值范围为 $0 \leq k < w$ 。设 x' 为 $w-k$ 位 $[x_{w-1}, x_{w-2}, \dots, x_k]$ 表示的补码数，而 x'' 为低 k 位 $[x_{k-1}, \dots, x_0]$ 表示的无符号数。通过与对无符号情况类似的分析，我们有 $x = 2^k x' + x''$ ，而 $0 \leq x'' < 2^k$ ，得到 $x' = \lfloor x/2^k \rfloor$ 。进一步，可以观察到，算术右移位向量 $[x_{w-1}, x_{w-2}, \dots, x_0]k$ 位，得到位向量

$$[x_{w-1}, \dots, x_{w-1}, x_{w-1}, x_{w-2}, \dots, x_k]$$

它刚好就是将 $[x_{w-1}, x_{w-2}, \dots, x_k]$ 从 $w-k$ 位符号扩展到 w 位。因此，这个移位后的位向量就是 $\lfloor x/2^k \rfloor$ 的补码表示。 ■

我们可以通过在移位之前“偏置(biasing)”这个值，来修正这种不合适的舍入。

原理：除以 2 的幂的补码除法，向上舍入

C 变量 x 和 k 分别有补码值 x 和无符号数值 k ，且 $0 \leq k < w$ ，则当执行算术移位时，C 表达式 $(x + (1 << k) - 1) >> k$ 产生数值 $\lfloor x/2^k \rfloor$ 。

图 2-30 说明在执行算术右移之前加上一个适当的偏置量是如何导致结果正确舍入的。在第 3 列，我们给出了 -12 340 加上偏量值之后的结果，低 k 位(那些会向右移出的位)以斜体表示。我们可以看到，低 k 位左边的位可能会加 1，也可能不会加 1。对于不需要舍入的情况($k=1$)，加上偏量只影响那些被移掉的位。对于需要舍入的情况，加上偏量导致较高的位加 1，所以结果会向零舍入。

k	偏量	$-12\,340 + \text{偏量}$	>> k (二进制)	十进制	$-12340/2^k$
0	0	1100111111001100	1100111111001100	-12340	-12340.0
1	1	1100111111001101	1110011111100110	-6170	-6170.0
4	15	11001111101011	1111110011111101	-771	-771.25
8	255	110100001001011	111111111010000	-48	-48.203125

图 2-30 补码除以 2 的幂(右移之前加上一个偏量，结果就向零舍入了)

偏置技术利用如下属性：对于整数 x 和 $y(y>0)$ ， $\lceil x/y \rceil = \lfloor (x+y-1)/y \rfloor$ 。例如，当 $x=-30$ 和 $y=4$ ，我们有 $x+y-1=-27$ ，而 $\lceil -30/4 \rceil = -7 = \lfloor -27/4 \rfloor$ 。当 $x=-32$ 和 $y=4$ 时，我们有 $x+y-1=-29$ ，而 $\lceil -32/4 \rceil = -8 = \lfloor -29/4 \rfloor$ 。

推导：除以 2 的幂的补码除法，向上舍入

查看 $\lceil x/y \rceil = \lfloor (x+y-1)/y \rfloor$ ，假设 $x=qy+r$ ，其中 $0 \leq r < y$ ，得到 $(x+y-1)/y = q + (r+y-1)/y$ ，因此 $\lfloor (x+y-1)/y \rfloor = q + \lfloor (r+y-1)/y \rfloor$ 。当 $r=0$ 时，后面一项等于 0，而当 $r>0$ 时，等于 1。也就是说，通过给 x 增加一个偏量 $y-1$ ，然后再将除法向下舍入，当 y 整除 x 时，我们得到 q ，否则，就得到 $q+1$ 。

回到 $y=2^k$ 的情况，C 表达式 $x+(1<<k)-1$ 得到数值 $x+2^k-1$ 。将这个值算术右移 k 位即产生 $\lfloor x/2^k \rfloor$ 。

这个分析表明对于使用算术右移的补码机器，C 表达式

```
(x<0 ? x+(1<<k)-1 : x) >> k
```

将会计算数值 $x/2^k$ 。

 **练习题 2.42** 写一个函数 `div16`，对于整数参数 x 返回 $x/16$ 的值。你的函数不能使用除法、模运算、乘法、任何条件语句 (`if` 或者 `?:`)、任何比较运算符 (例如 `<`、`>` 或 `==`) 或任何循环。你可以假设数据类型 `int` 是 32 位长，使用补码表示，而右移是算术右移。

现在我们可以看到，除以 2 的幂可以通过逻辑或者算术右移来实现。这也正是为什么大多数机器上提供这两种类型的右移。不幸的是，这种方法不能推广到除以任意常数。同乘法不同，我们不能用除以 2 的幂的除法来表示除以任意常数 K 的除法。

 **练习题 2.43** 在下面的代码中，我们省略了常数 M 和 N 的定义：

```
#define M      /* Mystery number 1 */
#define N      /* Mystery number 2 */
int arith(int x, int y) {
    int result = 0;
    result = x*M + y/N; /* M and N are mystery numbers. */
    return result;
}
```

我们以某个 M 和 N 的值编译这段代码。编译器用我们讨论过的方法优化乘法和除法。下面是将产生出的机器代码翻译回 C 语言的结果：

```
/* Translation of assembly code for arith */
int optarith(int x, int y) {
    int t = x;
    x <<= 5;
    x -= t;
    if (y < 0) y += 7;
    y >>= 3; /* Arithmetic shift */
    return x+y;
}
```

M 和 N 的值为多少？

2.3.8 关于整数运算的最后思考

正如我们看到的，计算机执行的“整数”运算实际上是一种模运算形式。表示数字的有限字长限制了可能的值的取值范围，结果运算可能溢出。我们还看到，补码表示提供了一种既能表示负数也能表示正数的灵活方法，同时使用了与执行无符号算术相同的位级实现，这些运算包括像加法、减法、乘法，甚至除法，无论运算数是以无符号形式还是以补码形式表示的，都有完全一样或者非常类似的位级行为。

我们看到了 C 语言中的某些规定可能会产生令人意想不到的结果，而这些结果可能是难以察觉或理解的缺陷的源头。我们特别看到了 `unsigned` 数据类型，虽然它概念上很简单，但可能导致即使是资深程序员都意想不到的行为。我们还看到这种数据类型会以出乎意料的方式出现，比如，当书写整数常数和当调用库函数时。

 **练习题 2.44** 假设我们在对有符号值使用补码运算的 32 位机器上运行代码。对于有符号值使用的是算术右移，而对于无符号值使用的是逻辑右移。变量的声明和初始化如下：

```
int x = foo();    /* Arbitrary value */
int y = bar();    /* Arbitrary value */

unsigned ux = x;
unsigned uy = y;
```

对于下面每个 C 表达式，1) 证明对于所有的 x 和 y 值，它都为真(等于 1)；或者 2) 给出使得它为假(等于 0)的 x 和 y 的值：

- A. $(x > 0) \parallel (x-1 < 0)$
- B. $(x \& 7) != 7 \parallel (x < 29 < 0)$
- C. $(x * x) >= 0$
- D. $x < 0 \parallel -x <= 0$
- E. $x > 0 \parallel -x >= 0$
- F. $x+y == uy+ux$
- G. $x \sim y + uy*ux == -x$

2.4 浮点数

浮点表示对形如 $V = x \times 2^y$ 的有理数进行编码。它对执行涉及非常大的数字($|V| \gg 0$)、非常接近于 0($|V| \ll 1$)的数字，以及更普遍地作为实数运算的近似值的计算，是很有用的。

直到 20 世纪 80 年代，每个计算机制造商都设计了自己的表示浮点数的规则，以及对浮点数执行运算的细节。另外，它们常常不会太多地关注运算的精确性，而把实现的速度和简便性看得比数字精确性更重要。

大约在 1985 年，这些情况随着 IEEE 标准 754 的推出而改变了，这是一个仔细制订的表示浮点数及其运算的标准。这项工作是从 1976 年开始由 Intel 赞助的，与 8087 的设计同时进行，8087 是一种为 8086 处理器提供浮点支持的芯片。他们请 William Kahan(加州大学伯克利分校的一位教授)作为顾问，帮助设计未来处理器浮点标准。他们支持 Kahan 加入一个 IEEE 资助的制订工业标准的委员会。这个委员会最终采纳的标准非常接近于 Kahan 为 Intel 设计的标准。目前，实际上所有的计算机都支持这个后来被称为 IEEE 浮点的标准。这大大提高了科学应用程序在不同机器上的可移植性。

旁注 IEEE(电气和电子工程师协会)

电气和电子工程师协会(IEEE，读做“eye-triple-ee”)是一个包括所有电子和计算机技术的专业团体。它出版刊物，举办会议，并且建立委员会来定义标准，内容涉及从电力传输到软件工程。另一个 IEEE 标准的例子是无线网络的 802.11 标准。

在本节中，我们将看到 IEEE 浮点格式中数字是如何表示的。我们还将探讨舍入(rounding)的问题，即当一个数字不能被准确地表示为这种格式时，就必须向上调整或者向下调整。然后，我们将探讨加法、乘法和关系运算符的数学属性。许多程序员认为浮点数没意思，往坏了说，深奥难懂。我们将看到，因为 IEEE 格式是定义在一组小而一致的原则上的，所以它实际上是相当优雅和容易理解的。